Oliver Krauss

Modern Code - Debugging & Error Analysis

ModernCode | MTD.ba VZ SS25

Hagenberg 15.12.2025

HAGENBERG | LINZ | STEYR | WELS

FH OBERÖSTERREICH

UNIVERSITY OF APPLIED SCIENCES UPPER AUSTRIA

Debugging & Error Analysis

- **Every programmer's reality**: Spending significant time debugging code
- **The challenge**: Code doesn't always work as expected
- **Real-world impact**: Debugging skills are essential for professional development
- **Connection to what you know**:
  - Methods (Session 04): Understanding call stack helps debug
  - Control structures (Session 03): Tracing loops to find bugs
  - Recursion (Session 10): Stack behavior affects debugging
  - Exception handling (Session 08): try-catch basics for error handling
- **The key question**: How do you systematically find and fix bugs?

Why This Matters

- **Time efficiency**: Good debugging skills save hours of frustration
- **Problem-solving ability**: Systematic debugging improves your coding skills
- **Professional readiness**: Debugging is a core skill in software development
- **Code quality**: Understanding errors helps you write better code
- **Confidence**: Knowing how to debug makes you a more capable programmer
- **Foundation for testing**: Debugging connects to writing tests that prevent bugs

Before We Begin: Think About This

What happens when your code doesn't work? What's your first step?

Have you ever seen an error message you didn't understand?

Before We Begin: Examples to Consider

Common Scenarios

- **Code compiles but crashes**: Program runs, then stops with an error
- **Code runs but gives wrong answer**: Everything seems fine, but result is incorrect
- **Code won't compile**: Red squiggles everywhere, can't even run it

Your Experience

- What debugging strategies have you tried?
- What confused you most about error messages?
- How do you currently find bugs in your code?

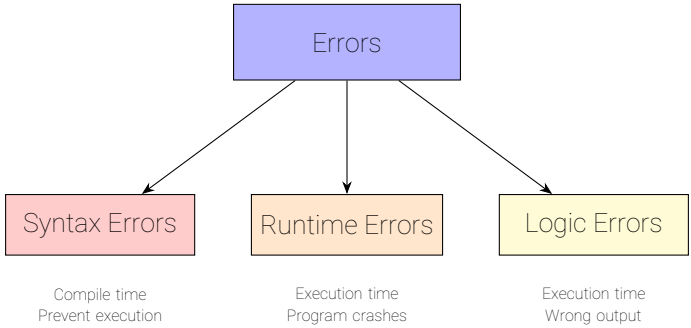
Today we'll learn systematic approaches to all of these!

Three Main Types of Errors

- **Syntax errors**: Code violates language rules → won't compile
- **Runtime errors**: Code compiles but crashes during execution
- **Logic errors**: Code runs but produces incorrect results

Each type requires different debugging strategies

Error Classification



Syntax Errors

- **Definition**: Code violates Java syntax rules
- **When caught**: At compile time (before program runs)
- **Visual indicator**: Red squiggles in IDE
- **Result**: Program won't compile or run

Common Examples

- Missing semicolons: `int x = 5` (should be `int x = 5;`)
- Typos: `publc` instead of `public`
- Mismatched brackets: `{ ... ( ... }`
- Missing quotes: `String s = Hello;`

Good news: Compiler tells you exactly where the error is!

Syntax Error Example

```
1   public class Example {
2       public static void main(String[] args) {
3           int x = 5
4           IO.println(x);
5       }
6   }
```

Compiler Error Message

```
/PATH/TO/FILE/Example.java:3:18
java:  ';' expected
```

The fix: Add semicolon after `int x = 5`

Understanding Compiler Error Messages

Error message format:

```
/PATH/TO/FILE/Example.java:3:18
java:  ';' expected
```

Breaking it down:

- `/PATH/TO/FILE/Example.java` → The file where the error occurred
- `:3:` → **Line number** where the error was detected
- `:18` → **Column/position** in that line (character position)
- `java:  ';' expected` → Description of what's wrong

Important

`Example.java:3:18` means:
Line 3, position 18 in the file `Example.java`

Tip: The position number tells you exactly where in the line the compiler got confused

Complex Syntax Errors: Missing Brackets

```
1   public class Example {
2       public static void main(String[] args) {
3           if (x > 5) {
4               IO.println("Large");
5           // Missing closing brace here!
6           IO.println("Done");
7       }
8   }
```

```
Example.java:8:1
java:  reached end of file while parsing
```

The error is NOT at line 8!

The compiler only realized something was wrong when it reached the end.

The actual problem: missing `}` after line 4 (and another errors is hidden)

How to Find the Real Problem

When the compiler says "end of file" or reports an error at the end:

1. **Check brackets**: Count opening `{` and closing `}`
   - Every `{` needs a matching `}`
   - Every `(` needs a matching `)`
   - Every `[` needs a matching `]`
2. **Check indentation**: Misaligned code often indicates missing brackets
3. **Start from the error location and work backwards**:
   - Look for unmatched brackets before the reported line
   - Check method/class declarations
4. **Use IDE bracket matching**: Most IDEs highlight matching brackets

If you see "reached end of file while parsing" or errors at the very end,
the problem is almost always a missing closing bracket somewhere earlier!

Runtime Errors

- **Definition**: Errors that occur while program is running
- **When caught**: During execution (after compilation)
- **Visual indicator**: Program crashes, error message appears
- **Result**: Program stops unexpectedly

Common Examples

- `NullPointerException`: Trying to use a null object
- `ArrayIndexOutOfBoundsException`: Accessing invalid array index
- `ArithmeticException`: Division by zero
- `NumberFormatException`: Converting invalid string to number

Connection: Exception handling (try ... catch) helps catch these

Runtime Error Example

```
1   public class Example {
2       public static void main(String[] args) {
3           int[] arr = {1, 2, 3};
4           IO.println(arr[5]);  // Runtime error!
5       }
6   }
```

Runtime Error Message

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException:  5
at Example.main(Example.java:3)
```

The problem: Array has 3 elements (indices 0-2), but we access index 5

Logic Errors

- **Definition**: Code runs but produces incorrect results
- **When caught**: During execution (but no crash)
- **Visual indicator**: Wrong output, unexpected behavior
- **Result**: Program appears to work but gives wrong answer

Common Examples

- Off-by-one errors: Loop runs one time too many/few
- Wrong formula: `area = width * width` instead of `width * height`
- Incorrect condition: `if (x > 5)` when should be `if (x >= 5)`
- Wrong operator: Using `+` instead of `*`

The challenge: These are hardest to find - program seems to work!

Logic Error Example

```
1   // Find maximum value in array
2   public static int findMax(int[] arr) {
3       int max = 0;  // Logic error!
4       for (int i = 0; i < arr.length; i++) {
5           if (arr[i] > max) {
6               max = arr[i];
7           }
8       }
9       return max;
10  }
```

The Problem

Logic Error: The Fix and Characteristics

The fix: Initialize `max = arr[0]` to handle arrays with all negative values

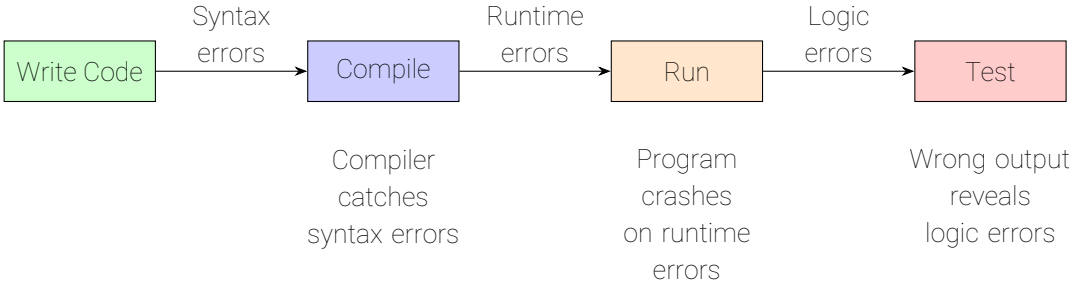
Logic Error Characteristics

- Code compiles successfully
- Program runs without crashing
- But produces **wrong results** due to flawed logic

Why This is a Logic Error

The code works correctly for arrays with positive numbers,
but fails when all values are negative. The algorithm logic is flawed,
not the syntax or runtime behavior.

When Errors Are Caught



Key insight: The earlier you catch an error, the easier it is to fix!

Error Types: Summary

Type	When Caught	Program Runs?	Example
Syntax	Compile time	No	Missing semicolon
Runtime	Execution time	Crashes	Array out of bounds
Logic	Execution time	Yes (wrong)	Off-by-one error

Try catch: Exception handling helps manage runtime errors gracefully

What is a Stack Trace?

```
Exception in thread "main" java.lang.NullPointerException
    at Calculator.divide(Calculator.java:15)
    at Calculator.calculate(Calculator.java:8)
    at Main.main(Main.java:5)
```

- **Definition**: A report showing the sequence of method calls that led to an error
- **Purpose**: Shows exactly where the error occurred and how we got there
- **Visual representation**: Like a breadcrumb trail through your code
- **Key insight**: Tells you the call chain from main() to the error

Why It Matters

The error might occur in one method, but the real problem could be in the calling method!

Reading a Stack Trace: The Basics

- **Read from top to bottom**: Most recent call first, oldest call last
- **Top line**: Where the error actually occurred
- **Below that**: How we got there (call chain)
- **Each line shows**: Method name, file name, line number

The Pattern

```
ExceptionType:  Error message
at ClassName.methodName(FileName.java:lineNumber)
```

Think of it as: "Error happened here, called from here, called from here..."

Stack Trace Example

```
Exception in thread "main" java.lang.NullPointerException
    at Calculator.divide(Calculator.java:15)
    at Calculator.calculate(Calculator.java:8)
    at Main.main(Main.java:5)
```

How to Read This

1. **Error type**: `NullPointerException`
2. **Where it happened**: `Calculator.divide()` at line 15
3. **Called from**: `Calculator.calculate()` at line 8
4. **Called from**: `Main.main()` at line 5

The call chain: `main()` → `calculate()` → `divide()` → ERROR

Stack Trace Anatomy

```
ExceptionType:  Error message
```

Exception type and descriptive message

```
at Class.method(File.java:15)
```

Where error occurred (most recent call)

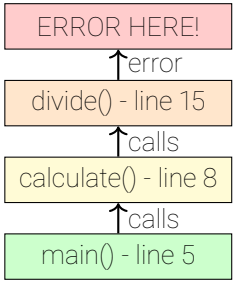
```
at Other.method(File.java:8)
```

Called from this method

```
at Main.main(Main.java:5)
```

Original entry point

Call Stack Visualization



Stack grows upward
Each method call adds a frame

Error at top (most recent)

Connection to recursion: Same stack behavior as recursion!

NullPointerException

What It Means

Trying to use a variable that is **`null`** (doesn't point to an object)

```
Exception in thread "main" java.lang.NullPointerException
    at Student.getName(Student.java:10)
    at Main.printStudent(Main.java:15)
    at Main.main(Main.java:5)
```

What to check:

- Line 10 in **`Student.java`**: What variable is null?
- Line 15 in **`Main.java`**: Did we pass null to **`printStudent()`**?
- Line 5 in **`Main.java`**: Did we create the Student object?

ArrayIndexOutOfBoundsException

What It Means

Trying to access an array index that doesn't exist

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 5
    at ArrayUtils.findMax(ArrayUtils.java:12)
    at Main.processData(Main.java:20)
    at Main.main(Main.java:8)
```

What to check:

- Line 12 in `ArrayUtils.java`: Index 5 is out of bounds
- What is the array size? (Valid indices are 0 to size-1)
- Line 20 in `Main.java`: What array was passed to `findMax()`?

Index problems: Arrays are zero-indexed!

StackOverflowError

What It Means

Too many recursive calls (or infinite recursion)

```
Exception in thread "main" java.lang.StackOverflowError
    at Factorial.factorial(Factorial.java:8)
    at Factorial.factorial(Factorial.java:8)
    at Factorial.factorial(Factorial.java:8)
    ... (many more lines)
```

What to check:

- Same method calls itself repeatedly
- Missing or incorrect base case
- Recursive call doesn't move toward base case

Recursion: This is why base cases are essential!

Practice: Reading a Stack Trace

Given This Stack Trace

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at Calculator.divide(Calculator.java:18)
    at Calculator.calculate(Calculator.java:10)
    at Main.processNumbers(Main.java:25)
    at Main.main(Main.java:7)
```

Questions to Answer:

1. What type of error occurred?
2. Where did it happen? (file and line)
3. What is the call chain?
4. What is likely the problem?

Practice: Reading a Stack Trace

Given This Stack Trace

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at Calculator.divide(Calculator.java:18)
    at Calculator.calculate(Calculator.java:10)
    at Main.processNumbers(Main.java:25)
    at Main.main(Main.java:7)
```

Questions to Answer:

1. What type of error occurred?
2. Where did it happen? (file and line)
3. What is the call chain?
4. What is likely the problem?

Answers:

1. `ArithmeticException` - division by zero
2. `Calculator.java:18` in `divide()` method
3. `main()` → `processNumbers()` → `calculate()` → `divide()`
4. Dividing by zero - check if denominator is 0

Focus on the Important Parts

Stack traces can be long - focus on what matters!

```
Exception in thread "main" java.lang.NullPointerException
    at MyClass.processData(MyClass.java:15)        // YOUR CODE - Important!
    at MyClass.main(MyClass.java:8)                // YOUR CODE - Important!
    at java.base/java.lang.Thread.run(Thread.java:1589)  // Framework - Ignore
    at java.base/java.util.concurrent...           // Framework - Ignore
    ... (many more framework lines)                // Framework - Ignore
```

Focus On

- **Top lines** - where error occurred
- **Your class names** - `MyClass`, `Calculator`, etc.
- **Your file names** - `MyClass.java`, `Main.java`
- **First few stack entries** - your call chain

Ignore

- **Framework code** - `java.base`, `java.util`
- **Library code** - third-party libraries
- **Bottom of stack** - usually framework internals
- **Very long traces** - focus on top 5-10 lines

Using AI to Understand Stack Traces

AI Can Help

- **Paste the stack trace**: "Explain this stack trace: [paste]"
- **Ask specific questions**: "Why is there a NullPointerException at line 15?"
- **Get suggestions**: "How do I fix this ArrayIndexOutOfBoundsException?"

Example Prompt:

> *"I'm getting this error: [stack trace]. Can you explain what's happening and where I should look to fix it?"*

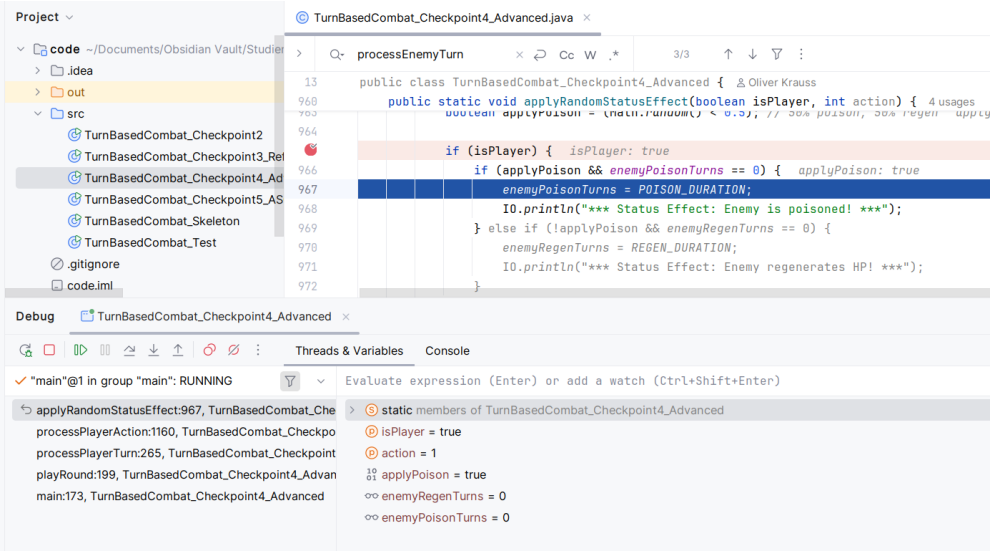
Remember: AI can explain, but you need to understand the concepts to fix the root cause!

What is a Debugger?

- **Definition**: A tool that lets you pause program execution and inspect state
- **Key features**:
  - Pause execution at specific points (breakpoints)
  - Step through code line by line
  - Inspect variable values
  - See the call stack
  - Evaluate expressions
- **Why use it**: Much more powerful than print statements
- **Visual advantage**: See exactly what's happening as code runs

Think of it as: A machine that lets you freeze your program and examine it

Debugger Example



Setting Breakpoints

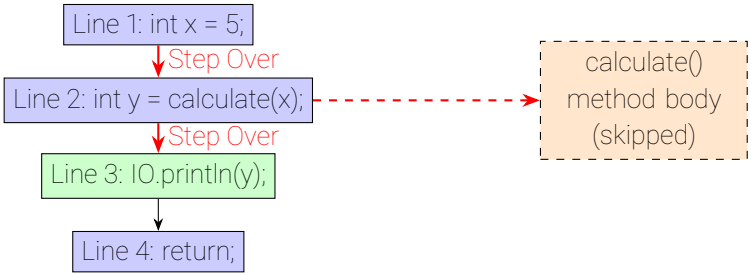
- **What is a breakpoint?**: A marker that tells the debugger to pause execution
- **How to set**: Click in the left margin next to a line of code
- **Visual indicator**: Red circle appears (IntelliJ)
- **When it triggers**: Program pauses before executing that line

Best Practices

- Set breakpoints at suspicious lines
- Set breakpoints at method entry points
- Set breakpoints before loops to inspect loop variables
- Use conditional breakpoints for specific values

Process: Set breakpoint, run in debug mode, program pauses automatically!

Debugging Controls: Step Over (F8)



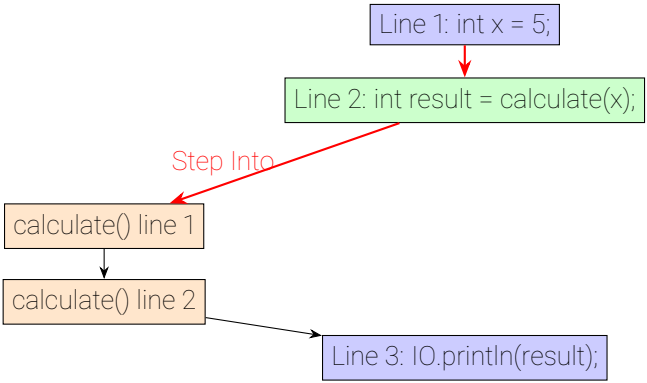
Step Over (F8)

Executes `calculate()` but skips going inside the method

Moves directly to next line

Use when: You want to execute a method call without debugging inside the method

Debugging Controls: Step Into (F7)

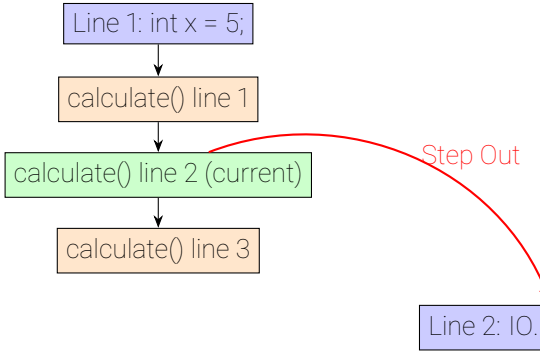


Step Into (F7)
Enters method calls to debug inside them

Use to trace method execution

Use when: You want to see what happens inside a method call

Debugging Controls: Step Out (Shift+F8)

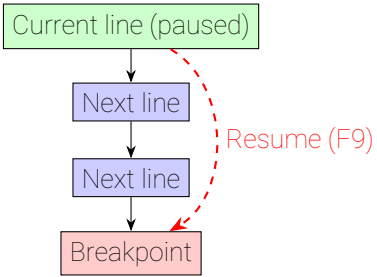


Step Out (Shift+F8)
Finishes current method
and returns to caller

Skip remaining method lines

Use when: You're done debugging inside a method and want to return

Debugging Controls: Resume (F9)



Resume (F9)
Continues execution until next breakpoint

Or until program finishes

Use when: You want to skip ahead to the next breakpoint

Inspecting Variables

Variables Panel

- Shows all variables in current scope
- Updates as you step through code
- Shows object fields
- Color-coded by type

Watches

- Monitor specific expressions
- Update automatically
- Useful for complex conditions

Evaluate Expression

- Test calculations on the fly
- Call methods with test values
- See results without modifying code

Example

- Variable: `arr.length`
- Watch: `i < arr.length`
- Evaluate: `arr[i] * 2`

Key insight: See exactly what values variables have at any point!

Call Stack View in Debugger

- **Visual representation**: Shows the call chain from main() to current method
- **Interactive**: Click any method to see its variables
- **Connection**: Same information as stack trace, but interactive
- **Useful for**: Understanding how you got to the current point

Example Call Stack View

- `main()` (line 5) - can click to see main's variables
- `processData()` (line 20) - can click to see processData's variables
- `calculate()` (line 10) - current method, variables visible

Finding the Problem: Navigate up the call stack to see what values were passed to methods

Debugging Example: Factorial Method

```
1   static int factorial(int n) {
2       if (n <= 1) {
3           return 1;
4       }
5       return n * factorial(n - 1);
6   }
```

1. Set breakpoint at line 2 (if condition)
2. Run in debug mode with `factorial(4)`
3. Step through to see:
   - First call: n = 4, condition false, goes to line 4
   - Recursive call: n = 3, condition false, goes to line 4
   - Continue until n = 1, condition true, returns 1
4. Watch call stack grow and shrink

Debugging Loops

- **Set breakpoint**: Inside the loop or at loop condition
- **Step through**: See how loop variables change each iteration
- **Watch variables**: Monitor loop counter, array index, accumulator
- **Use Resume**: Skip iterations you don't need to see

Example: Debugging a Sum Loop

- Breakpoint at: `sum += arr[i];`
- Watch: `i`, `arr[i]`, `sum`
- Step through: See sum accumulate
- Identify: Off-by-one errors, wrong indices

Debugging Recursive Calls

- **Watch call stack**: See it grow with each recursive call
- **Inspect parameters**: See how n changes with each call
- **Step into**: Enter recursive calls to see base case reached
- **Watch return values**: See values propagate back up

Example: Debugging factorial(4)

- Call 1: n = 4, stack has 1 frame
- Call 2: n = 3, stack has 2 frames
- Call 3: n = 2, stack has 3 frames
- Call 4: n = 1, stack has 4 frames, base case!
- Returns: Watch stack shrink as values return

Connection to Recursion: Visualize the recursive call stack!

Conditional Breakpoints

- **What**: Breakpoint that only triggers when condition is true
- **When to use**:
  - Loop runs many times, but you only care about specific iteration
  - Only want to pause when variable has certain value
  - Debugging specific case in large dataset

Example

- Breakpoint condition: `i == 10`
- Only pauses on 10th loop iteration
- Saves time skipping first 9 iterations

How it works: Right-click breakpoint to set condition in IntelliJ

Conditional Breakpoints: Practical Example I

Problem: Debugging a loop that processes 1000 items, but error only occurs at item 847

```
1  public static void processArray(int[] data) {
2      for (int i = 0; i < data.length; i++) {
3          if (data[i] < 0) {  // Set conditional breakpoint here
4              IO.println("Negative value at index " + i);
5              // Error occurs when i == 847
6          }
7      }
8  }
```

Conditional Breakpoints: Practical Example II

Without Condition

- Breakpoint triggers 1000 times
- Must click "Resume" 846 times
- Very time-consuming!

With Condition

- Set condition: `i == 847`
- Breakpoint triggers only once
- Immediately at the problem!

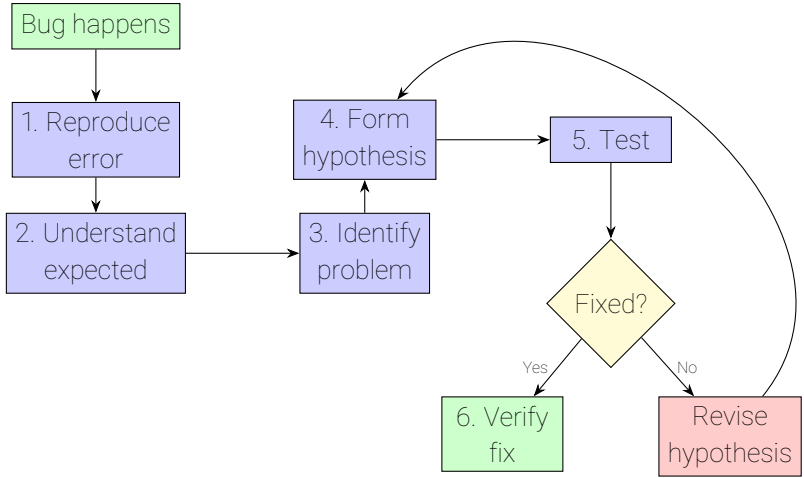
Other Useful Conditions

`data[i] < 0` - pause only for negative values

`i > 100 && i < 200` - pause in specific range

`name.equals("test")` - pause for specific string

Systematic Debugging Approach



Step 1: Reproduce the Error

- **Why it matters**: Can't debug what you can't see
- **What to do**:
  - Run the code that causes the error
  - Note the exact steps to trigger it
  - Capture the error message
  - Document the input that causes it

If You Can't Reproduce It

- Check if it's intermittent (happens sometimes)
- Look for specific conditions (certain input, timing)
- Ask: What's different when it works vs when it fails?

Key insight: Consistent reproduction makes debugging much easier

Step 2: Understand What Should Happen

- **Define expected behavior**: What should the code do?
- **Check requirements**: Does the code match the specification?
- **Review logic**: Does the algorithm make sense?
- **Compare with similar code**: How do similar programs work?

Questions to Ask

- What is the correct output for this input?
- What should each variable contain at each step?
- What is the expected flow of execution?

Tip: Write down expected behavior before debugging - helps identify deviations

Step 3: Identify Where It Goes Wrong

- **Use stack trace**: Points to exact line of error
- **Use debugger**: Step through to see where behavior diverges
- **Use print statements**: Add debug output to trace execution
- **Binary search**: Test middle of code, narrow down location

Divide and Conquer Strategy

1. Test first half of suspicious code - does it work?
2. If yes, problem is in second half
3. If no, problem is in first half
4. Repeat with smaller section

Key insight: Isolate the problem to a small section of code

Step 4: Form Hypothesis

- **Based on evidence**: Use what you've observed
- **Be specific**: "Variable x is null" not "something is wrong"
- **Consider common patterns**: Is this a known error type?
- **Think about root cause**: Not just the symptom

Good Hypotheses

- "The loop condition is wrong, causing off-by-one error"
- "The array index calculation is incorrect"
- "The variable is null because it wasn't initialized"

Good hypothesis: "string comparison uses == instead of .equals(), so it sometimes returns false"

Step 5: Test Hypothesis

- **Check the hypothesis**: Use debugger or print statements
- **Verify assumptions**: Is variable really null? Is index really wrong?
- **Test with different inputs**: Does hypothesis hold for other cases?
- **Look for evidence**: Does data support your hypothesis?

Testing Methods

- Inspect variable values in debugger
- Add print statements to verify values
- Modify code temporarily to test fix
- Use evaluate expression in debugger

Key insight: Don't assume - verify with evidence!

Step 6: Fix and Verify

- **Apply the fix**: Make the code change
- **Test the fix**: Run the code again
- **Verify it works**: Check with original failing case
- **Test edge cases**: Make sure fix doesn't break other cases

Verification Checklist

- Original error is fixed
- Other test cases still work
- No new errors introduced
- Code is still readable

Important: Fix the root cause, not just the symptom!

Print Debugging (printf Debugging)

- **What**: Adding print statements to see what's happening
- **When to use**:
  - Quick debugging without debugger
  - Understanding program flow
  - Checking variable values
  - Tracing execution path

What to Print

- Variable values at key points
- "Entering method X" messages
- Loop iteration numbers
- Condition evaluations

Print Debugging Example

Before (No Debug)

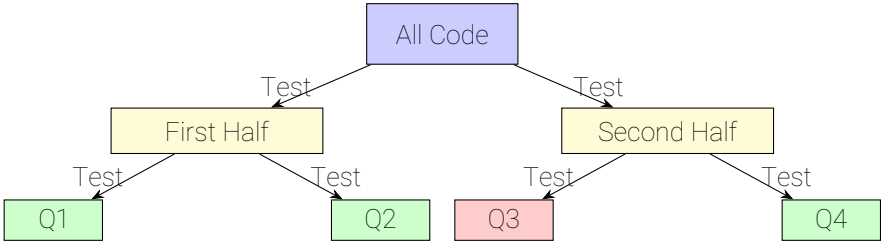
```
1    int sum = 0;
2    for (int i = 0; i < arr.length; i++) {
3        sum += arr[i];
4    }
5    return sum;
```

After (With Debug)

```
1    int sum = 0;
2    IO.println("Starting sum, arr.length=" + arr.length)
         ;
3    for (int i = 0; i < arr.length; i++) {
4        IO.println("i=" + i + ", arr[i]=" + arr[i]);
5        sum += arr[i];
6        IO.println("sum=" + sum);
7    }
8    IO.println("Final sum=" + sum);
9    return sum;
```

Output shows: Exact values at each step, making bugs obvious

Divide and Conquer Debugging



Problem in Q3!
Narrowed down to 25% of code

Strategy: Test parts separately to isolate the problem

Rubber Duck Debugging

- **What**: Explaining your code to someone (or something) else
- **Why it works**: Forces you to think through logic step by step
- **How**:
  - Explain what code should do
  - Explain what it actually does
  - Explain line by line
  - Often you'll find the bug while explaining!

The Process

1. Get a rubber duck (or person, or AI)
2. Explain your code to it
3. As you explain, you'll notice inconsistencies
4. The bug becomes obvious!

Reading Code Carefully

- **Trace execution**: Follow the code line by line mentally
- **Check conditions**: What makes each condition true/false?
- **Track variables**: How do variables change?
- **Verify logic**: Does the logic match the intent?

Common Mistakes to Avoid

- Assuming code does what you think it does
- Skipping over "obvious" parts
- Not checking edge cases
- Ignoring error messages

Tip: Read code out loud - helps catch mistakes

Common Debugging Mistakes to Avoid

Mistake 1: Assuming Instead of Checking

- **Don't**: "The variable must be 5"
- **Do**: Check the variable value in debugger

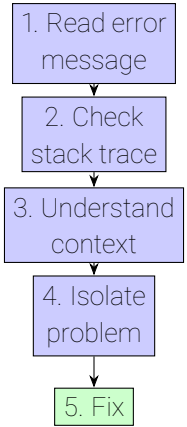
Mistake 2: Not Reading Error Messages

- **Don't**: Ignore the error message
- **Do**: Read it carefully - it tells you what's wrong!

Mistake 3: Fixing Symptoms Not Causes

- **Don't**: Fix the immediate error without understanding why
- **Do**: Find and fix the root cause

Systematic Error Analysis



Key insight: Follow a systematic process, don't guess randomly

Step 1: Read the Error Message Carefully

- **Don’t skip it**: Error messages contain valuable information
- **Read completely**: Every word matters
- **Look for keywords**: Exception type, file name, line number
- **Understand the message**: What is it telling you?

Example Error Message

```
java.lang.ArrayIndexOutOfBoundsException:  5
at MyClass.processArray(MyClass.java:12)
```

Information extracted:

- Error type: ArrayIndexOutOfBoundsException
- Problematic index: 5
- Location: MyClass.java, line 12, processArray method

Step 2: Check the Stack Trace

- **Start from top**: Most recent call first
- **Follow the chain**: See how you got to the error
- **Identify the source**: Where did the bad data come from?
- **Check each level**: Problem might be in calling method

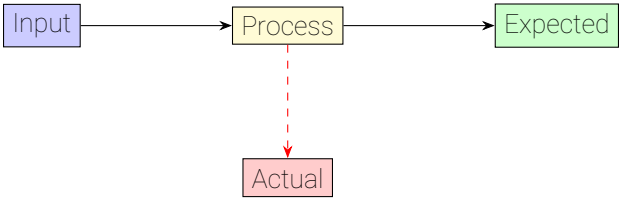
Stack Trace Analysis

- Top line: Where error occurred (symptom)
- Lower lines: How we got there (might reveal cause)
- Look for patterns: Same method called multiple times?

Connection to Section 3: Use stack trace reading skills here!

Step 3: Understand the Context

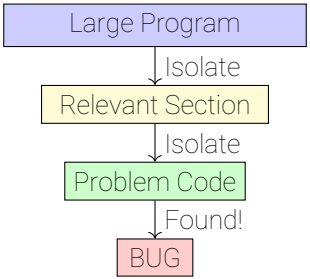
- **What was the input?**: What data caused the error?
- **What was expected?**: What should have happened?
- **What actually happened?**: What did occur?
- **When does it happen?**: Always or sometimes?



Key insight: Context helps identify why the error occurred

Step 4: Isolate the Problem

- **Minimal reproducible example**: Smallest code that shows the bug
- **Remove unrelated code**: Focus on the problem area
- **Test in isolation**: Run just the problematic part
- **Simplify inputs**: Use simplest data that triggers error



Methods: Smaller code is easier to debug and understand

Error Message Interpretation

Breaking Down an Error Message

```
java.lang.NullPointerException
at Calculator.divide(Calculator.java:15)
```

Components:

- `java.lang`: Package
- `NullPointerException`: Error type
- `Calculator.divide`: Method name
- `Calculator.java`: File name
- `15`: Line number

What It Tells You:

- Something is null
- Error at line 15
- In divide method
- Check that line!

Action: Go to Calculator.java, line 15, look for null variable

Using Test Cases for Analysis

- **Test with known inputs**: Use values you understand
- **Test edge cases**: Boundary values, empty inputs
- **Compare results**: Expected vs actual
- **Identify pattern**: When does it work? When does it fail?

Input	Expected	Actual
0	1	1 (OK)
1	1	1 (OK)
5	120	120 (OK)
-1	Error	1 (WRONG!)

Pattern identified: Negative numbers not handled correctly

Minimal Reproducible Example

What is it?

- The smallest piece of code that still shows the bug
- Contains only what's necessary to reproduce the error
- Removes all unrelated code, methods, and complexity

Why do you need it?

- **Show someone else the bug**: When you can't figure it out yourself
- **Keep it minimal for them**: Others can help faster with less code
- **Easier to understand**: Focus on the problem, not the surrounding code

How to create it: Remove code until bug disappears, then add back last removed part

Minimal Reproducible Example

Original (Complex)

```
public class Complex {
    // 50 lines of code
    public void process() {
        // 30 lines
        int result = calculate();
        // 20 lines
    }
    // More methods...
}
```

Minimal (Simple)

```
public class Test {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3};
        int x = arr[5];  // Bug!
    }
}
```

Goal: Create the minimal code that still has the bug

Process: Remove code until bug disappears, then add back last removed part

What is Testing?

- **Definition**: Verifying that code works correctly
- **Purpose**:
  - Catch bugs early
  - Prevent regressions (bugs coming back)
  - Document expected behavior
  - Give confidence in code changes
- **Key insight**: Tests prove your code does what it should

Testing Philosophy

"If it's not tested, it's broken"

Tests are your safety net when making changes

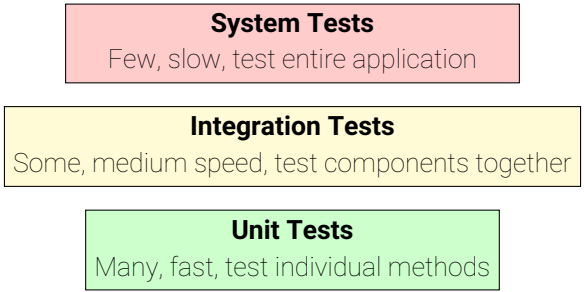
Why Test?

- **Catch bugs early**: Find problems before users do
- **Prevent regressions**: Ensure fixes don't break other things
- **Document behavior**: Tests show how code should work
- **Enable refactoring**: Change code confidently with tests
- **Save time**: Automated tests run faster than manual testing
- **Improve design**: Writing tests forces you to think about design

Real-world impact: Professional developers write tests for their code

Testing Pyramid

Note:
Integration & System tests covered next semester

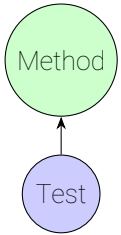


Workload: More unit tests, fewer integration tests, fewest system tests

Three Levels of Testing: Overview

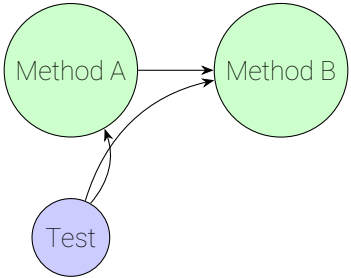
Unit Tests

- Test one method
- Fast execution
- Many tests
- Isolated



Integration Tests

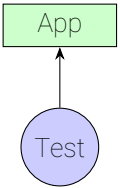
- Test components together
- Medium speed
- Some tests
- Test interactions



(Overview - next semester)

System Tests

- Test entire app
- Slow execution
- Few tests
- End-to-end



(Overview - next semester)

Unit Testing: What to Test

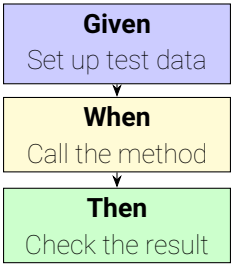
- **Individual methods**: Test one method at a time
- **Different inputs**: Normal cases, edge cases, boundary cases
- **Expected outputs**: Verify method returns correct result
- **Isolation**: Test method independently of others

Example: Testing factorial()

- Test with n = 0 (edge case)
- Test with n = 1 (edge case)
- Test with n = 5 (normal case)
- Test with n = -1 (error case)

Key insight: Test one thing at a time, with various inputs

GWT Pattern (Given-When-Then)



The Pattern:

1. **Given**: Prepare inputs, set up test data (preconditions)
2. **When**: Call the method being tested (the action)
3. **Then**: Verify the result is correct (the assertion)

GWT Pattern Example

```
1   // Given: Set up test data (preconditions)
2   int input = 5;
3   int expected = 120;
4
5   // When: Call the method (the action)
6   int result = factorial(input);
7
8   // Then: Check the result (the assertion)
9   assertEquals(expected, result);
```

- Clear separation of concerns
- Easy to read and understand
- Standard pattern used everywhere
- Natural language: "Given X, when Y, then Z"

Multiple Names for Same Concept

Different Names, Same Structure

- **GWT**: Given-When-Then (used here, common in BDD)
- **AAA**: Arrange-Act-Assert (also very common)
- **Setup-Exercise-Verify**: Another variation
- All describe the same three-part test structure!

GWT	AAA	Setup-Exercise-Verify
Given	Arrange	Setup
When	Act	Exercise
Then	Assert	Verify

Key insight: The concept is the same, only the names differ!

Test Cases: Types

Normal Cases

- Typical inputs that should work
- Example: `factorial(5)` should return 120

Edge Cases

- Boundary values, empty inputs, single elements
- Example: `factorial(0)`, `factorial(1)`

Error Cases

- Invalid inputs that should be handled
- Example: `factorial(-1)` should throw exception or return error

All are important: Test all three types to ensure robustness

Test Case Table Example

Input	Expected	Type
0	1	Edge case
1	1	Edge case
5	120	Normal case
10	3628800	Normal case
-1	Error	Error case

Assertions: How to Check Results

- **assertEquals(expected, actual)**: Check if values are equal
- **assertTrue(condition)**: Check if condition is true
- **assertFalse(condition)**: Check if condition is false
- **assertNull(value)**: Check if value is null
- **assertNotNull(value)**: Check if value is not null

Example Assertions

```
1    assertEquals(120, factorial(5));
2    assertTrue(isValidEmail("test@example.com"));
3    assertFalse(isValidEmail("invalid"));
4    assertNull(getUserById(999));  // User doesn't exist
```

Key insight: Assertions are how tests verify correctness

Example: Testing with GWT Pattern

Normal Case

```
@Test
void testFactorialNormal() {
    // Given: Set up test data
    int input = 5;
    int expected = 120;

    // When: Call the method
    int result = factorial(input);

    // Then: Verify the result
    assertEquals(expected, result);
}
```

Edge Case

```
@Test
void testFactorialEdgeCase() {
    // Given: Edge case input
    int input = 0;
    int expected = 1;

    // When: Call the method
    int result = factorial(input);

    // Then: Verify edge case result
    assertEquals(expected, result);
}
```

Key points:

- Both tests follow GWT structure
- Each test focuses on one scenario
- Consistent structure makes tests easy to read

Writing Good Unit Tests

Best Practices

- **Test one thing**: Each test should verify one behavior
- **Use descriptive names**: `testFactorialWithZero` not `test1`
- **Test edge cases**: Don't just test normal cases
- **Keep tests simple**: Easy to read and understand
- **Test independently**: Tests shouldn't depend on each other

What to Avoid

- Testing multiple things in one test
- Vague test names
- Only testing happy path
- Complex test setup

Good vs Bad Test Examples

BAD

```
@Test
void test1() {
    int x = factorial(5);
    int y = factorial(0);
    int z = sum(arr);
    // Tests 3 things!
    // Name is meaningless
}
```

GOOD

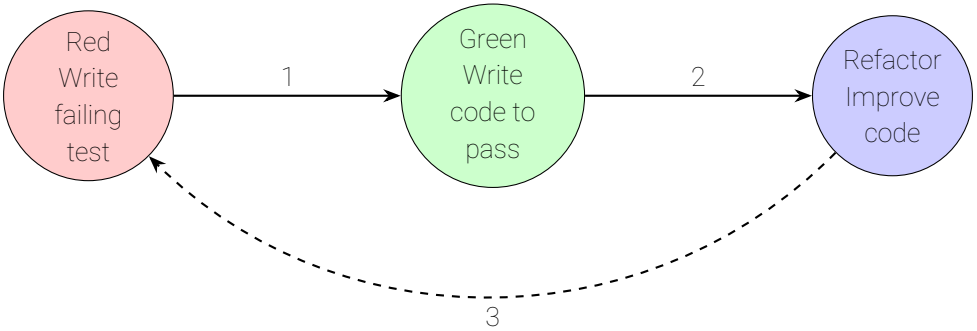
```
@Test
void testFactorialWithFive() {
    // Given
    int input = 5;
    int expected = 120;

    // When
    int result = factorial(input);

    // Then
    assertEquals(expected, result);
}
```

Key insight: Good tests are clear, focused, and easy to understand

Test-Driven Development (TDD) Overview



The TDD Cycle:

1. **Red**: Write a test that fails (code doesn't exist yet)
2. **Green**: Write minimal code to make test pass
3. **Refactor**: Improve code while keeping tests green
4. Repeat

Running Tests in IntelliJ

- **Visual indicators**: Green checkmark (pass), red X (fail)
- **Run single test**: Click green arrow next to test method
- **Run all tests**: Click green arrow next to class
- **Test results panel**: Shows which tests passed/failed
- **Failure details**: Click failed test to see error message

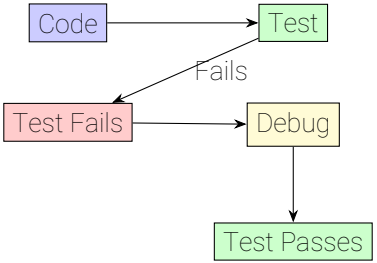
Test Runner Interface

- Green: All tests passed
- Red: Some tests failed
- Shows test execution time
- Lists all test methods

Test early: Run tests frequently to catch bugs early

Connection: Testing and Debugging

- **Tests find bugs**: Failed test points to problem
- **Tests guide debugging**: Know exactly what's broken
- **Tests verify fixes**: Confirm bug is actually fixed
- **Tests prevent regressions**: Ensure fixes don't break other things



Testing is a bottom-up approach: test small components first, then build up.
Debugging often uses bottom-up strategies (divide and conquer, isolate problems), making them complementary approaches to ensuring code quality.

What is Profiling?

- **Definition**: Measuring program performance to find bottlenecks
- **Purpose**:
  - Identify slow parts of code
  - Find performance bottlenecks
  - Optimize where it matters
  - Compare different implementations
- **Key insight**: Don't guess what's slow - measure it!

Profiling Philosophy

"Measure, don't guess"
Profiling shows you where time is actually spent

Why Profile?

- **Find bottlenecks**: Know which methods take longest
- **Optimize effectively**: Focus on slow parts, not fast ones
- **Compare solutions**: Which implementation is faster?
- **Verify optimizations**: Did your changes actually help?
- **Avoid premature optimization**: Don't optimize what doesn't need it

Real-world impact: Performance matters in production systems

"**Premature optimization is the root of all evil.**" - Donald Knuth

IntelliJ Profiler Basics

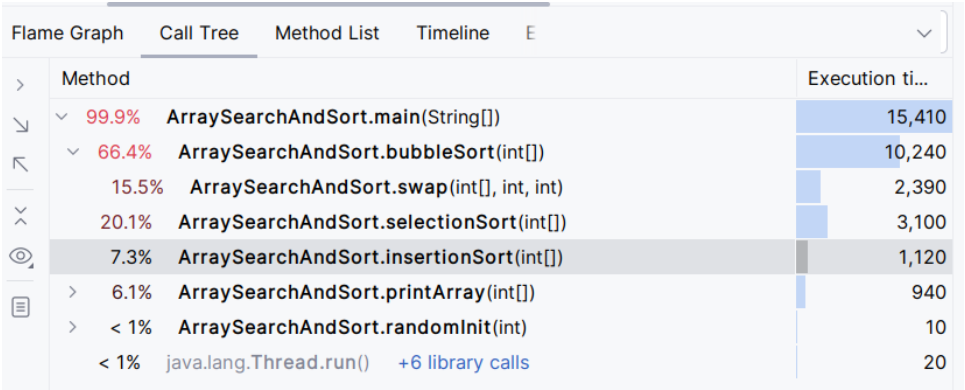
- **How to start**: Run program with profiler enabled
- **CPU profiling**: See which methods use most CPU time
- **Memory profiling**: See memory usage (basic overview)
- **Visual interface**: Graphs and tables showing performance data

Profiler Output Shows

- Method execution times
- Call counts
- Hot spots (slow methods)
- Call tree visualization

Performance: Profiler helps identify methods that need optimization

Profiler Interface: Call Tree View



- **Call Tree**: Shows method hierarchy and execution times
- **Execution time**: Time spent in each method (and children)
- **Visual bars**: Quick visual comparison of method performance

Timing Code Manually

- **System.currentTimeMillis()**: Returns current time in milliseconds
- **System.nanoTime()**: More precise, returns nanoseconds
- **Process**: Record time before, after, calculate difference

When to Use Manual Timing

- Quick performance checks
- Comparing two implementations
- Simple timing needs
- Learning about performance

Note: Profiler is better for complex analysis, but manual timing is simpler

Manual Timing Example I

```
long startTime = System.currentTimeMillis();

// Code to measure
int result = factorial(20);

long endTime = System.currentTimeMillis();
long duration = endTime - startTime;

IO.println("Execution time: " + duration + " ms");
```

Manual Timing Example II

Using nanoTime for Precision

```
long start = System.nanoTime();
// Code here
long end = System.nanoTime();
double durationMs = (end - start) / 1_000_000.0;
IO.println("Time: " + durationMs + " ms");
```

How to measure: Measure before and after, calculate difference

Example: Timing Sorting Algorithms

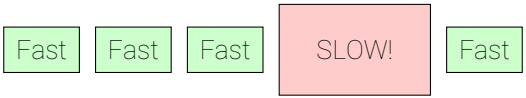
Comparing Different Sorts

- Time bubble sort on array of 1000 elements
- Time merge sort on same array
- Compare results
- See which is faster

Algorithm	Time (ms)
Bubble Sort	150
Merge Sort	5

Identifying Bottlenecks

- **What is a bottleneck?**: Part of code that slows down entire program
- **How to find**: Profiler shows which methods take longest
- **Hot spots**: Methods that consume most execution time
- **80/20 rule**: Often 20% of code takes 80% of time

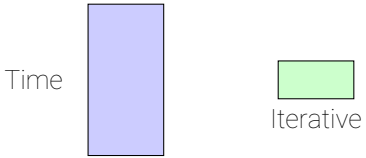


Key insight: Optimize the bottleneck, not the fast parts

Performance Comparison

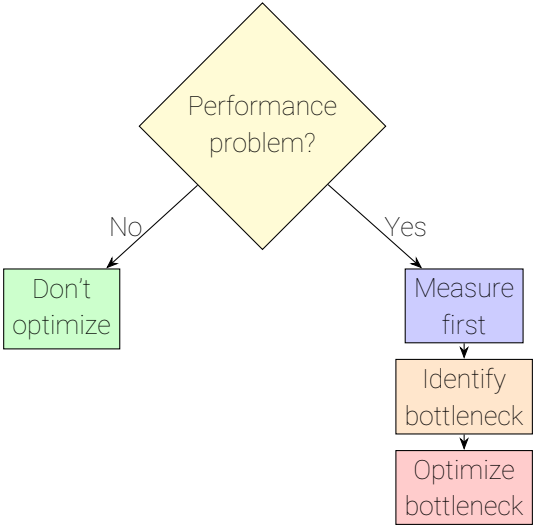
- **Compare implementations**: Test different approaches
- **Measure multiple times**: Average results for accuracy
- **Use same input**: Fair comparison requires same data
- **Visualize results**: Charts make differences obvious

Factorial(20)



Iterative is much faster!

When to Optimize



Key principles:

- Don't optimize prematurely
- Measure first, then optimize
- Optimize bottlenecks, not everything

Remember

Profiling tells you
where to optimize,
not *when* to optimize

Common Performance Issues

Inefficient Algorithms

- Recursion when iteration is possible (most times)
- Nested loops when single loop works
- Example: Bubble sort vs merge sort

Unnecessary Computations

- Recalculating same value multiple times
- Not caching results
- Example: Computing factorial(5) multiple times

Key insight: Profiling reveals these issues so you can fix them

Before and After Optimization

BEFORE (Slow)

```
// Computing factorial(5)
// in every loop iteration!
for (int i = 0; i < 1000; i++) {
    result = factorial(5) * i;
    // factorial(5) computed
    // 1000 times!
}
```

AFTER (Fast)

```
// Compute once, reuse result!
int fact5 = factorial(5);
for (int i = 0; i < 1000; i++) {
    result = fact5 * i;
    // factorial(5) computed
    // only once!
}
```

Profiling showed: `factorial(5)` was computed 1000 times
Optimization: Compute once before loop, reuse the result
Result: Much faster - same calculation, computed once instead of 1000 times

Complete Debugging Example: Sorting Bug

The Problem

Sorting algorithm doesn't sort correctly - some elements are out of order

Step-by-step debugging:

1. Reproduce: Run with test array, see wrong output
2. Read error: No exception, but wrong result (logic error)
3. Set breakpoint: Inside sorting loop
4. Step through: Watch array elements change
5. Identify: Comparison logic is wrong
6. Fix: Correct the comparison
7. Verify: Test again, output is correct

Debugging Example: Sorting Bug

```
1     void bubbleSort(int[] arr) {
2         for (int i = 0; i < arr.length; i++) {
3             for (int j = 0; j < arr.length - 1; j++) {
4                 if (arr[j] < arr[j + 1]) {  // BUG: Should be >
5                     int temp = arr[j];
6                     arr[j] = arr[j + 1];
7                     arr[j + 1] = temp;
8                 }
9             }
10        }
11    }
```

- Set breakpoint at line 4 (if statement)
- Watch: arr[j] and arr[j+1] values
- Notice: Comparison is backwards
- Fix: Change **<** to **>**

Debugging Example: Recursive Function

The Problem

Recursive factorial returns wrong value for some inputs

Debugging steps:

1. Set breakpoint at method entry
2. Call with factorial(4)
3. Watch call stack grow: factorial(4) $\rightarrow$ factorial(3) $\rightarrow$ ...
4. Inspect parameter n at each level
5. Watch return values propagate back
6. Identify: Base case returns wrong value
7. Fix: Correct base case
8. Verify: Test with multiple inputs

Debugging Example: Recursive Bug

```
1   static int factorial(int n) {
2       if (n <= 1) {
3           return 0;  // BUG: Should return 1
4       }
5       return n * factorial(n - 1);
6   }
```

- factorial(4) calls factorial(3)
- factorial(3) calls factorial(2)
- factorial(2) calls factorial(1)
- factorial(1) returns 0 (WRONG!)
- All results become 0

The fix: Change `return 0` to `return 1`

Best Practices Summary

Debugging Checklist

- **Read error messages**: They tell you what's wrong
- **Use the debugger**: Don't just guess
- **Set breakpoints**: Pause at suspicious locations
- **Inspect variables**: See actual values
- **Step through code**: Understand execution flow
- **Check stack trace**: Understand call chain
- **Test systematically**: Don't make random changes
- **Verify fixes**: Make sure problem is actually solved

Systematic Patient Thorough

Debugging Tools Summary

IDE Tools

- Breakpoints
- Step controls
- Variable inspection
- Call stack view
- Evaluate expression

Key insight: Use the right tool for the situation

Manual Techniques

- Print debugging
- Stack trace analysis
- Code reading
- Test cases
- Rubber duck debugging

NullPointerException

- **What causes it**: Trying to use a variable that is `null`
- **Common scenarios**:
  - Object not initialized before use
  - Method returns null unexpectedly
  - Array element is null

Variable
points to
nothing

Valid
object
reference

How to prevent: Always check for null or initialize variables

NullPointerException Example

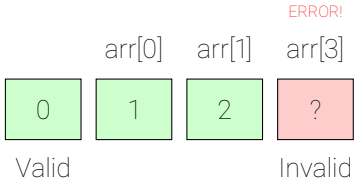
```
1  String name = null;
2  int length = name.length();  // NullPointerException!
```

The Fix (is often defensive programming!)

```
1  String name = null;
2  if (name != null) {
3      int length = name.length();
4  } else {
5      IO.println("Name is null!");
6  }
```

ArrayIndexOutOfBoundsException

- **What causes it**: Accessing array index that doesn't exist
- **Valid indices**: 0 to (length - 1)
- **Common mistakes**:
  - Using length as index: `arr[arr.length]`
  - Off-by-one in loops: `for (int i = 0; i <= arr.length; i++)`
  - Negative indices: `arr[-1]`



ArrayIndexOutOfBoundsException Example

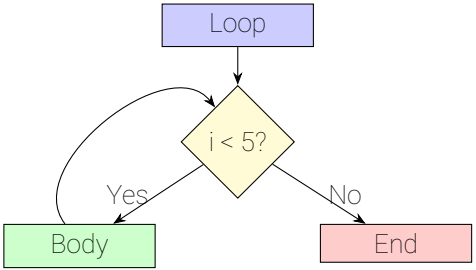
```
int[] arr = {10, 20, 30};  // indices: 0, 1, 2
IO.println(arr[3]);  // Error! Index 3 doesn't exist
```

The Fix (is again defensive programming)

```
int[] arr = {10, 20, 30};
int index = 3;
if (index >= 0 && index < arr.length) {
    IO.println(arr[index]);
} else {
    IO.println("Index out of bounds!");
}
```

Off-by-One Errors

- **What causes it**: Loop runs one time too many or too few
- **Common patterns**:
  - Using `<=` instead of `<`
  - Starting at 1 instead of 0
  - Using `length` instead of `length - 1`



Off-by-one:
Loop runs wrong number of times

Off-by-One Error Example

WRONG

```
1    // Want to print 0-4
2    for (int i = 0; i <= 5; i++) {
3        IO.println(i);
4    }
5    // Prints: 0,1,2,3,4,5
6    // (6 numbers, not 5!)
```

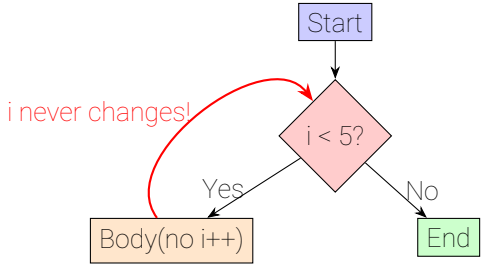
CORRECT

```
1    // Want to print 0-4
2    for (int i = 0; i < 5; i++) {
3        IO.println(i);
4    }
5    // Prints: 0,1,2,3,4
6    // (5 numbers, correct!)
```

The fix: Use **<** instead of **<=**, or adjust the bound

Infinite Loops

- **What causes it**: Loop condition never becomes false
- **Common causes**:
  - Forgetting to update loop variable
  - Wrong condition (always true)
  - Update moves away from termination



Infinite!
Condition
always true

Infinite Loop Example

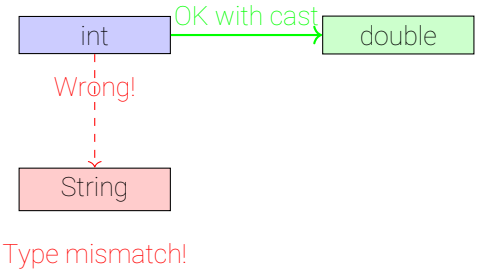
```
int i = 0;
while (i < 5) {
    IO.println(i);
    // BUG: Missing i++!
}
```

The Fix

```
int i = 0;
while (i < 5) {
    IO.println(i);
    i++;  // FIXED: Update loop variable
}
```

Type Mismatches

- **What causes it**: Using wrong data type
- **Common scenarios**:
  - Integer division when expecting decimal
  - String concatenation instead of addition
  - Wrong type in method call



Connection to Session 02: Understand type compatibility and casting

Type Mismatch Example

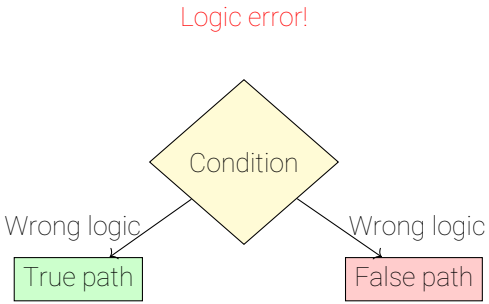
```
int a = 5;
int b = 2;
double result = a / b;  // Result is 2.0, not 2.5!
```

The Fix

```
int a = 5;
int b = 2;
double result = (double) a / b;  // Result is 2.5
```

Logic Errors in Conditions

- **What causes it**: Wrong boolean logic in conditions
- **Common mistakes**:
  - Using `AND` (&&) instead of `OR` (||) (or vice versa)
  - Wrong comparison operator (`>` vs `>=`)
  - Negation errors (`!` in wrong place)



Logic Error in Condition Example

```
int age = 18;
if (age > 18) {  // Logic error!
    IO.println("Adult");
} else {
    IO.println("Minor");
} // Prints "Minor" even though 18 is adult age
```

The Fix

```
int age = 18;
if (age >= 18) {  // FIXED: Use >=
    IO.println("Adult");
} else {
    IO.println("Minor");
}
```

Summary: Key Concepts

Error Types

- Syntax: Compile-time errors
- Runtime: Execution crashes
- Logic: Wrong results

Stack Traces

- Read top to bottom
- Shows call chain
- Points to error location

Debugging Tools

- Breakpoints
- Step controls
- Variable inspection
- Call stack view

Debugging Strategies

- Systematic approach
- Print debugging
- Divide and conquer

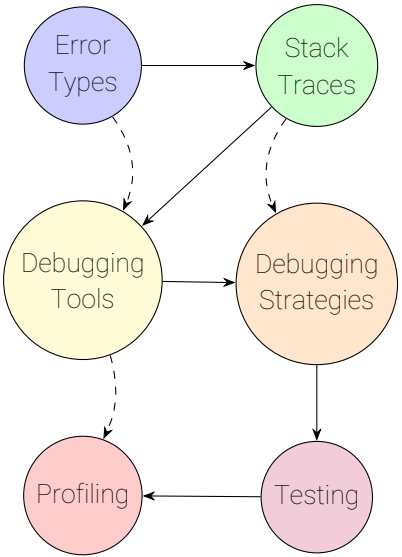
Testing

- Unit tests (detailed)
- Integration tests (overview)
- System tests (overview)
- GWT pattern

Profiling

- Measure performance
- Find bottlenecks
- Compare implementations
- Manual timing

Concept Map: Debugging & Error Analysis



How concepts connect:

- **Solid arrows**: Main learning flow
  - Understand errors → Read traces → Use tools → Apply strategies → Test → Profile
- **Dashed arrows**: Cross-connections
  - Error types help with tools
  - Traces inform strategies
  - Tools enable profiling

Key Insight

All concepts work together to help you find and fix bugs effectively